

Travaux Pratiques #1 : Architecture de Microservices

Contexte général

La banque WillBank dispose depuis plusieurs années d'une application monolithique de gestion des comptes et des opérations pour ses clients particuliers. Avec l'ouverture de nouvelles agences, l'ajout d'une application mobile et l'augmentation du nombre de transactions (salaire, mobile money, paiements marchands...), l'application devient :

- lente aux heures de pointe (fin de mois, paye des salaires),
- difficile à faire évoluer,
- peu flexible pour intégrer de nouveaux canaux (mobile, web, SMS, push notifications).

La DSI décide de migrer progressivement vers une **architecture de microservices**, afin de bénéficier de :

- la **montée en charge** (scalabilité horizontale),
- la **résilience** (pannes localisées),
- l'ouverture à de nouveaux canaux (mobile app, API partenaires),
- l'utilisation de **messagerie asynchrone (RabbitMQ)** et de **push notifications (Firebase Cloud Messaging)**.

Microservices de la 1ère version

Pour la première version de l'application bancaire, on définit les microservices suivants :

1. Microservice pour la gestion des clients :
 - a. Gestion des clients : création, mise à jour, KYC simplifié (nom, prénom, adresse, téléphone, email, statut, etc.).
2. Microservice pour la gestion des comptes :
 - a. Gestion des comptes bancaires :
 - i. comptes courants, comptes épargne,
 - ii. soldes, statuts (actif/bloqué).
3. Microservice pour la gestion des opérations/transactions:
 - a. Gestion des transactions:

- i. virements internes WillBank (compte à compte),
 - ii. dépôts, retraits, **paiements marchands**,
 - iii. historique des transactions.
- 4. Microservice pour la gestion des Notifications
 - a. Notifications asynchrones/synchrones :
 - i. Email (validation de compte, alerte de transaction, modification de profil),
 - ii. Push notifications via Firebase (FCM) pour l'app mobile : alerte en temps réel à chaque transaction.
 - b. Ce service consomme les événements RabbitMQ et déclenche les envois de notifications appropriés.
- 5. Microservice composite
 - a. Service composite qui expose des APIs "orientées business"
 - i. consultation du **dashboard client** : liste des comptes + solde + dernières transactions,
 - ii. relevé d'un compte sur une période,
 - iii. recherche des transactions par type (paiement, virement, retrait...).
 - b. Ce service agrège les données via des appels REST aux microservices : Client, Compte, Transaction.
- 6. **Les Microservices API Gateway & UI Service**
 - **API Gateway** : point d'entrée unique (authentification, routage, rate limiting, agrégation simple).
 - **UI Service** :
 - application web (CRM) et mobile (client) consommant l'API Gateway.

Objectifs pédagogiques

À l'issue du TP, l'étudiant doit être capable de :

1. **Concevoir une architecture microservices** dans un domaine bancaire.
2. **Définir les frontières d'un microservice.**
3. Concevoir :

- les **modèles de données** (chaque service avec sa base),
 - les **APIs REST** de chaque microservice.
4. Mettre en place :
- un **serveur de découverte** (Eureka),
 - une **API Gateway**,
 - un **broker de messages** (RabbitMQ),
 - un **service de notifications** utilisant **Firestore Cloud Messaging**.
5. Comprendre les échanges :
- **synchrone** (REST entre microservices),
 - **asynchrone** (événements RabbitMQ).

Part 1 : Dossier de Conception (Cahier de Conception)

1. Proposer l'architecture microservice de l'application.
2. Proposer pour chaque microservice, son architecture logique, son modèle de base de données, et la liste des APIs qu'il offre.

Part 2 : Codage

3. Implémenter chaque microservices de votre architecture, n'oublier pas l'API Gateway & le serveur de découverte éventuellement.

Par groupe de 3 : délais **11 Décembre 2025**.

Adresse à ajouter à votre dépôt : w.kungne.enspy@gmail.com

Deux séances de suivi et **notées** avant la présentation.